

Throughout modern society, there are samples of speech used to communicate, whether it be a recorded lecture, a music track, or even a phone conversation. And along with the speech in these samples, there can be noise that makes it more difficult to understand what the sample is trying to say. From background noise to erroneous glitches within the data that introduce random noise, these all can interfere with and distort the sound waves of the speech itself, rendering it inaudible. There are several ways that this can be filtered to allow for the speech itself to be recognizable, including bandpass filtering, high filtering, low filtering, and others, depending on the source and noise involved.

This report examines one of these methods, namely the bandpass filter, to extract speech from “noisy” .wav files that have various levels of background noise, and then compares the results with the matching “clean” .wav file. For this, the files were read into the script as matching pairs, pairing the same source “noisy” file with its “clean” counterpart. Then, to ensure that the samples all had the same known sample rate, they were resampled to a 16 kHz sampling rate. Afterwards, the “noisy” samples were run through a bandpass filter with the low edge at 300 Hz and the high edge at 3400 Hz. This range roughly corresponds to the range of human speech, though it might cut off some of the deeper base or higher soprano tones. The specific filter used was a 4th order butterworth filter, which provides a smooth and clean frequency response while minimizing the distortion of the speech signal. This filtered sample was then saved as a new .wav file, following the same naming format of the original samples to allow for direct comparison, tagging it as the filtered file. Then, the filtered sample was processed using spectrum

subtraction to help filter out background conversational noise, such as one might expect at a restaurant or train station, which would be missed by the initial bandpass filter. This involved several steps, starting with taking a fast Fourier transformation of the file, computing the full sample power spectrum, then the power spectrum of the background noise, subtracting this background noise from the FFT of the waveform, then reconstructing it with an inverse FFT. This also was saved as a new .wav file, also following the same nomenclature as the previous samples, tagged as the enhanced sample. Then, all four samples' waveforms were plotted on a single plot in a column, which was saved as a .png image for further analysis, evaluation, and extrapolation. All these files were then sorted into folders by the sample identifier, such that all 5 files, the clean, filtered, enhanced, and noisy .wav files and the graph .png file, for each sample, were together for ease of analysis.

The output files, including both the .wav files and the graphs, tend to show significant improvement overall to the original noisy file through the two filters applied. With the files that used more of a background conversational noise, while I was not able to completely filter it out, the filtering process did very well in making the foreground voice much more audible and understandable, essentially highlighting the voice while minimizing this background chatter. The files that used other methods of creating noise tended to come out of the bandpass filter very clean, though there is decidedly a change in the foreground voice, where it loses some of its tone. Overall, none of the files were able to completely match the "clean" samples, however, they were much closer to it and were very easy to understand what the speaker was saying.

The dataset used for this project is the ‘test’ data from the Valentini Noisy dataset on Kaggle.com (<https://www.kaggle.com/datasets/muhmagdy/valentini-noisy>). This set contains 824 files each of ‘noisy’ and ‘clean’ .wav files, paired with each other, using two different primary speakers. They are all named either p232_ for the male speaker or p257_ for the female speaker, then just listed by three-digit numbers, from 001 up to 415 for the male and 434 for the female, though it does not have every single number in between. The full set also includes a separate training set that can be used to train a machine learning model, with a total of 70.9 thousand files, all paired off in much the same way as this test data is.

Overall, this project took several iterations to achieve decent results. At first, just the bandpass filter was applied, which for most of the files worked just fine. After further examining the results, however, there were several that the bandpass filter made worse than the original, and these all had background conversational noise. The bandpass filter flattened the foreground speaker, so it tended to blend in more with the background. From there, I attempted to apply a Voice Activity Detector mask filter to help eliminate this conversational noise, however, this did very little to bring out the foreground speaker. From there, I learned that I could apply the spectral subtraction, which basically takes a sample of the background noise in the first quarter of a second before the foreground speaker starts to talk, and then uses that to help eliminate that noise from the rest of the sample. This, however, resulted in almost no audio left in the file at first, until I looked at the graph’s measurement – instead of having an amplitude for the resultant waveform maxing out at 1, it was more to the tune of 10^{-4} , a very small value. So, this was then normalized, which

once again brought out the speech, this time much clearer than either the ‘noisy’ or ‘filtered’ samples, and much closer to the ‘clean’ sample.

Overall, this showed how easy it is to take an audio recording and clean it up such that the main portion of the audio is much clearer and easier to understand. There could still be a few other filtering algorithms applied to this dataset to clean it up even more while still maintaining the quality of sound, however, as a base project on understanding the process, this shows the concept quite effectively.

Sample Results:





